Chapter 2

Client-Server Model & Request Lifecycle

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

Every system you will ever design in an interview is a variation of one idea: a **client** asks for something, a **server** does the work and answers back. This is the client-server model, and it is the single most important mental model in all of system design — almost everything else (load balancers, caches, databases) exists to make this one exchange faster, more reliable, or able to handle more clients at once.

A **client** is anything that initiates a request: a mobile app, a browser, another backend service. A **server** is anything that listens for requests and returns a response. Critically, "server" doesn't mean one physical machine — in real systems it's usually a fleet of machines behind a load balancer, but from the client's point of view, it still behaves like a single logical server.

> **Backend engineer analogy**
>
> This is literally the Laravel/Express request lifecycle you already know: a browser sends an HTTP request, it hits your router, your controller runs, it may query MySQL or hit Redis, and a response goes back. System design just zooms out: instead of one server, imagine 500 servers, and instead of one client, imagine 50 million phones. The lifecycle is identical in shape — only the scale changes.

The full request lifecycle, step by step

When you type a URL and press Enter (a question interviewers love because it touches almost every Phase 1 topic), roughly this happens:

1. **DNS resolution:** the browser converts the domain name (e.g., api.swiggy.com) into an IP address (covered in depth in Chapter 3).
2. **TCP connection:** the client and server perform a three-way handshake to establish a reliable connection (Chapter 4).
3. **TLS handshake (if HTTPS):** client and server agree on encryption keys so the connection is secure.
4. **HTTP request sent:** the client sends a method (GET/POST/etc.), headers, and optionally a body.
5. **Server processing:** the request hits a load balancer, is routed to an application server, which may query a database or cache.
6. **HTTP response sent back:** status code, headers, and body (HTML, JSON, etc.) return to the client.
7. **Client renders / consumes the response.**

In an HLD interview, you rarely need to describe every one of these steps in detail — but you need to know they exist, because interviewers will drill into whichever step is relevant to the problem (e.g., "what happens if the TCP connection drops mid-upload?").

A subtlety worth internalizing early: the client-server model itself says nothing about how many machines are behind "server." Whether the request lands on a monolith running on one box or a mesh of 40 microservices, the client's experience — send a request, get a response — is identical. This is precisely why interviewers can ask you to "design X" without specifying microservices vs. monolith upfront: the client-server contract at the edge doesn't change based on that internal decision, so you're free to justify either based on the problem's actual needs rather than defaulting to microservices because it sounds more advanced.

One more nuance worth flagging now, because it resurfaces constantly in later chapters: this lifecycle describes a single request. Real user sessions involve dozens of these round trips in sequence — load the home screen, fetch recommendations, load an image, submit a search — and each one independently goes through DNS lookup (usually cached after the first), a connection (often reused via HTTP keep-alive rather than re-negotiated every time), and the request/response cycle. Understanding that connections can be reused, not re-established from scratch every single time, is part of why real systems feel fast despite this seemingly heavy multi-step lifecycle.

2. Real-World Example / Case Study

When you open the PhonePe app to pay a merchant, the app (client) sends a request to PhonePe's backend (server) to initiate a transaction. That backend is not one machine — it's dozens of services behind load balancers, but the app only ever "talks" to what looks like a single API endpoint. The app doesn't know or care whether the request lands on server #12 or #340; that routing decision is made for it. This abstraction — many servers pretending to be one — is what almost every later chapter (load balancing, sharding, caching) is built to support.

Careem's ride-booking flow is another clean example: the rider's app (client) sends a booking request; Careem's backend (server-side, many services) matches a driver, and the response streams back updates over time. Note this already hints at a limitation of plain request-response: booking status changes *after* the initial response, which is why real-time systems need more than basic client-server calls (WebSockets, covered in Chapter 4 and again in Phase 3).

A third useful example: Meesho's seller dashboard. When a seller updates a product's stock count, the browser (client) sends a straightforward synchronous request, the server validates and writes the change, and returns success or failure — a textbook, low-drama use of the client-server model with no real-time requirement at all. Not every feature needs sophistication; recognizing when plain request-response is genuinely *sufficient* is just as much an HLD skill as knowing when it isn't. Interviewers notice when a candidate reaches for WebSockets or queues everywhere out of habit rather than need.

3. Diagram

```
CLIENT                                SERVER (logical - may be many machines)
  |                                          |
  |------ 1. DNS lookup ------------------->|  (resolves domain to IP)
  |<----- IP address returned -------------|
  |                                          |
  |------ 2. TCP handshake (SYN/SYN-ACK/ACK) ------------->|
  |                                          |
  |------ 3. HTTP Request (GET /orders) -->|
  |                                          |---> App logic
  |                                          |---> Query DB / Cache
  |<----- 4. HTTP Response (200, JSON) ----|
  |                                          |
[ Browser renders response ]
```

4. Trade-offs Table

Pattern	When to use	Limitation
Plain request-response (HTTP)	Simple CRUD, most REST APIs	Server can't push updates to client on its own
Polling (client asks repeatedly)	Near-real-time needs, simple to build	Wasteful; wastes requests when nothing changed
Long polling	Better than polling, moderate real-time needs	Still holds connections open; harder to scale
WebSockets (full-duplex)	True real-time: chat, live tracking	More complex infra; covered fully in Phase 3

5. Common Interview Questions

Q1: "Walk me through what happens when a user opens your app's home screen."

Model approach: Narrate the lifecycle top to bottom — DNS, connection, request, load balancer, app server, cache/DB lookup, response. Don't over-explain DNS/TCP in depth unless asked; spend more time on what happens once the request reaches your system, since that's what the interviewer actually cares about.

Q2: "Is HTTP good enough for a chat application? Why or why not?"

Model approach: Explain that plain HTTP is client-initiated only, so the server can't push a new message to a client without the client asking. Mention polling and long-polling as stopgaps, then correctly name WebSockets as the standard solution, briefly noting the trade-off (persistent connections cost server resources at scale).

Q3: "What happens if the TCP connection drops mid-request?"

Model approach: Explain that the client typically detects the failure via a timeout or connection-reset error, and that the application layer must decide how to react — retry the whole request, resume from a checkpoint (common for large file uploads, using range requests), or surface an error to the user. Mention that idempotency (can this request be safely retried without side effects?) becomes critical here — a concept Phase 2, Chapter 9 covers in depth.

6. Practice Exercises

1. Write out, in your own words, the 7-step request lifecycle from memory, without looking back at this chapter.
2. For a food delivery app, identify at least one feature that fits plain request-response (e.g., viewing a menu) and one that does not (e.g., live delivery-partner location) and explain why.
3. Sketch the diagram from this chapter but replace "SERVER" with 3 separate boxes representing a load balancer and two app servers — a preview of Chapter 9.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Web Application Architecture: Full Request-Response Lifecycle"	Walks through the complete lifecycle end to end.
English	"Client-Server Architecture Explained: Request-Response Model"	Clean, visual explanation of the core model.
Hindi	"Client Server model in Hindi"	Direct Hindi explanation of the client-server concept.
Hindi	"Client Server Model (Distributed Systems, Hindi)"	Covers the same model with a distributed-systems framing.

8. Key Takeaways

- Client-server is the foundational pattern underneath every system you'll design in this program.
- "Server" is a logical concept — in production it's usually many machines behind a load balancer.
- The full request lifecycle: DNS → TCP handshake → (TLS) → HTTP request → server processing → HTTP response.
- Plain HTTP is client-initiated; the server cannot push data on its own.
- Polling, long-polling, and WebSockets exist specifically to work around that limitation for real-time needs.
- Interviewers use "what happens when you open the app" as a way to probe how deep your systems knowledge goes — go as deep as the conversation invites.